

SENAI FATESG

Análise e Desenvolvimento de Sistemas — 3º Semestre
Arquitetura e Projeto de Software

ADR-003

Generics, Mappers e Tratamento Global de Exceções

Status: **Em vigor** **Data:** Aula 03 **Autor:** Equipe

Caio Abreu | Cassiano Abreu | Gabriel Naoki | Wyllian Mariano
SENAI FATESG — Goiânia, GO — 01 de abril de 2025

Contexto

Com múltiplos domínios implementados (Cliente, Veículo, Serviço, Ordem de Serviço), ficou evidente que o CRUD básico era idêntico em todos eles — cada domínio reescrevia as mesmas operações de listar, buscar por ID, criar, atualizar e deletar no Controller e no Service. Além disso, a API não possuía um contrato padronizado para respostas de erro, retornando exceções brutas do Spring ao frontend Angular, dificultando o tratamento no lado do cliente.

Decisão

Introduzir classes genéricas no pacote core usando Java Generics para parametrizar por tipo de entidade, eliminando a duplicação de CRUD entre os domínios. Criar um `GlobalExceptionHandler` com `@RestControllerAdvice` para tratamento centralizado de todas as exceções. Adotar DTOs obrigatoriamente em todas as respostas da API, com Mappers dedicados para conversão entre Model e DTO.

Classes genéricas criadas:

<code>GenericController<M, D, S, MP></code>	→ CRUD completo por herança
<code>GenericService<M, R, V></code>	→ Orquestração com hook <code>beforeInsert</code>
<code>GenericValidation<M></code>	→ Validações reutilizáveis
<code>GenericMapper<M, D></code>	→ Conversão Model <-> DTO
<code>IGenericRepository<M></code>	→ Estende <code>JpaRepository</code>

Justificativa

- Generics eliminam duplicação de código CRUD — cada domínio herda o comportamento sem reescrever nada
- DTOs desacoplam o contrato da API da estrutura interna das entidades JPA — mudanças no banco não quebram o frontend
- O `GlobalExceptionHandler` garante que todos os erros retornem no mesmo formato (`ErrorResponse`), facilitando o tratamento no Angular
- A camada de Validation separada do Service segue o princípio de responsabilidade única (SRP)
- O hook `beforeInsert` no `GenericService` permite customização por domínio sem quebrar o fluxo genérico
- Interfaces para cada camada (`IGenericService`, `IGenericMapper`, etc.) garantem desacoplamento e facilitam testes

Alternativas Consideradas

- Copiar e colar o CRUD em cada domínio — descartado por gerar retrabalho, inconsistências e dificultar manutenção
- `MapStruct` ou `ModelMapper` para mapeamento — descartado para manter o projeto simples e didático, com mapeamento explícito
- Retornar entidades JPA diretamente na API — descartado por expor estrutura interna e criar acoplamento forte com o banco
- Tratamento de erro individual por controller — descartado por gerar duplicação e inconsistência nas respostas

Consequências

- Novos domínios ficam muito mais simples de implementar — apenas herdam as classes genéricas e sobrescrevem o necessário
- O contrato de erro padronizado (ErrorResponse) estabiliza a integração com o frontend Angular
- Aumenta o número de classes por domínio (interface + implementação para cada camada), exigindo disciplina da equipe
- A camada genérica cobre 80% dos casos — casos especiais ainda precisam de implementação específica por domínio
- Nunca expor entidades JPA diretamente nas respostas da API — regra obrigatória do projeto a partir desta versão